PATRONES ESTRUCTURALES · GOF

Composite

Tratar igual a un elemento simple y a un grupo de elementos



Brayan Caro - Deimis Henao - Luis Vega

¿Qué es Composite y de dónde viene?

QUÉ ES · PATRÓN ESTRUCTURAL

Compone objetos en estructuras de árbol y permite tratar igual a un objeto simple y a un grupo de objetos.

Interfaz común
Hojas y grupos comparten el mismo tipo, así que el cliente no distingue entre ellos.

El grupo guarda hijos
Su lista es del tipo común: puede contener hojas u otros grupos.

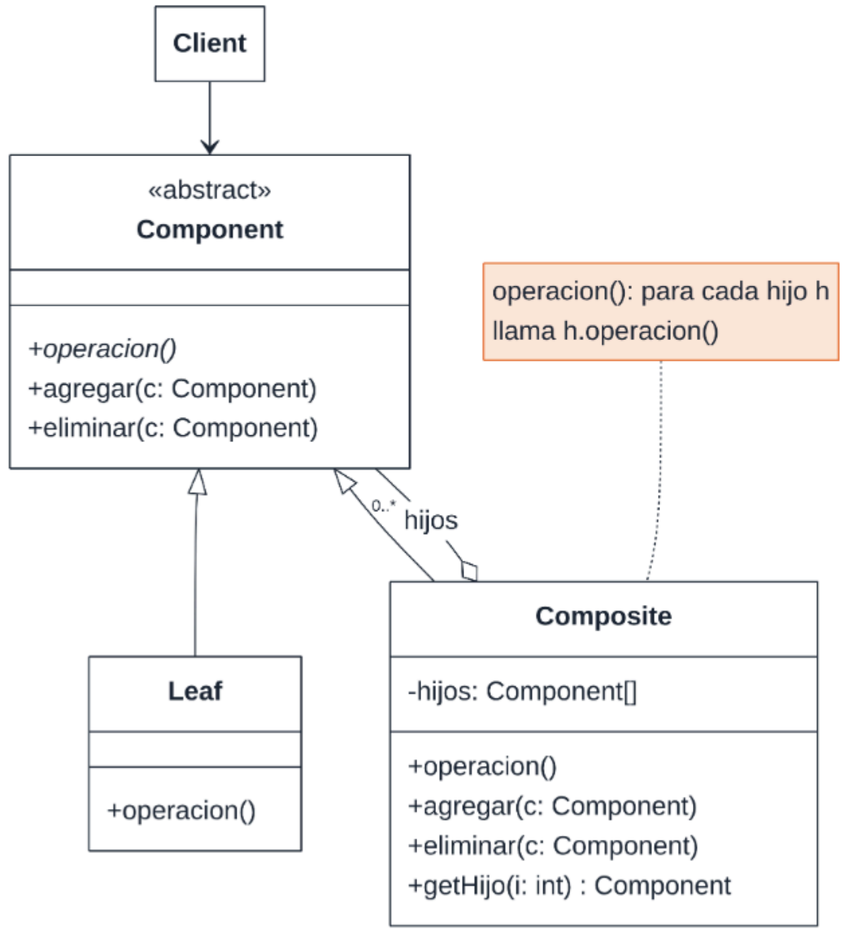
Delega y se repite
Le pide la operación a cada hijo; si el hijo es un grupo, hace lo mismo (recursión).

DE DÓNDE VIENE

1994	Libro Design Patterns: Elements of Reusable Object-Oriented Software.
GoF	Lo catalogaron Gamma, Helm, Johnson y Vlissides, la «Banda de los Cuatro».
Tipo	Patrón estructural: organiza cómo se combinan clases y objetos.
Ejemplo	Editores gráficos: un grupo de figuras contiene otros grupos. También se le llama Object Tree.

ESTRUCTURA DEL PATRÓN

Los 4 roles de Composite



Component

Clase abstracta o interfaz común. Dice **qué** se le puede pedir a cualquier elemento del árbol.

Leaf (hoja)

Elemento simple, sin hijos. Tiene el dato real y responde directo.

Composite

Grupo que guarda una lista de **Component**. Responde preguntándole a sus hijos.

Client

Usa todo a través de Component, sin saber si es hoja o grupo.

¿Qué problema resuelve?

Datos con forma de **árbol**: elementos simples y grupos que contienen otros grupos.

Plan → Semestre → Materias y Electivas → Materias

Sin tipo común: lista de Object

El cliente pregunta con instanceof

Los mismos if en cada operación

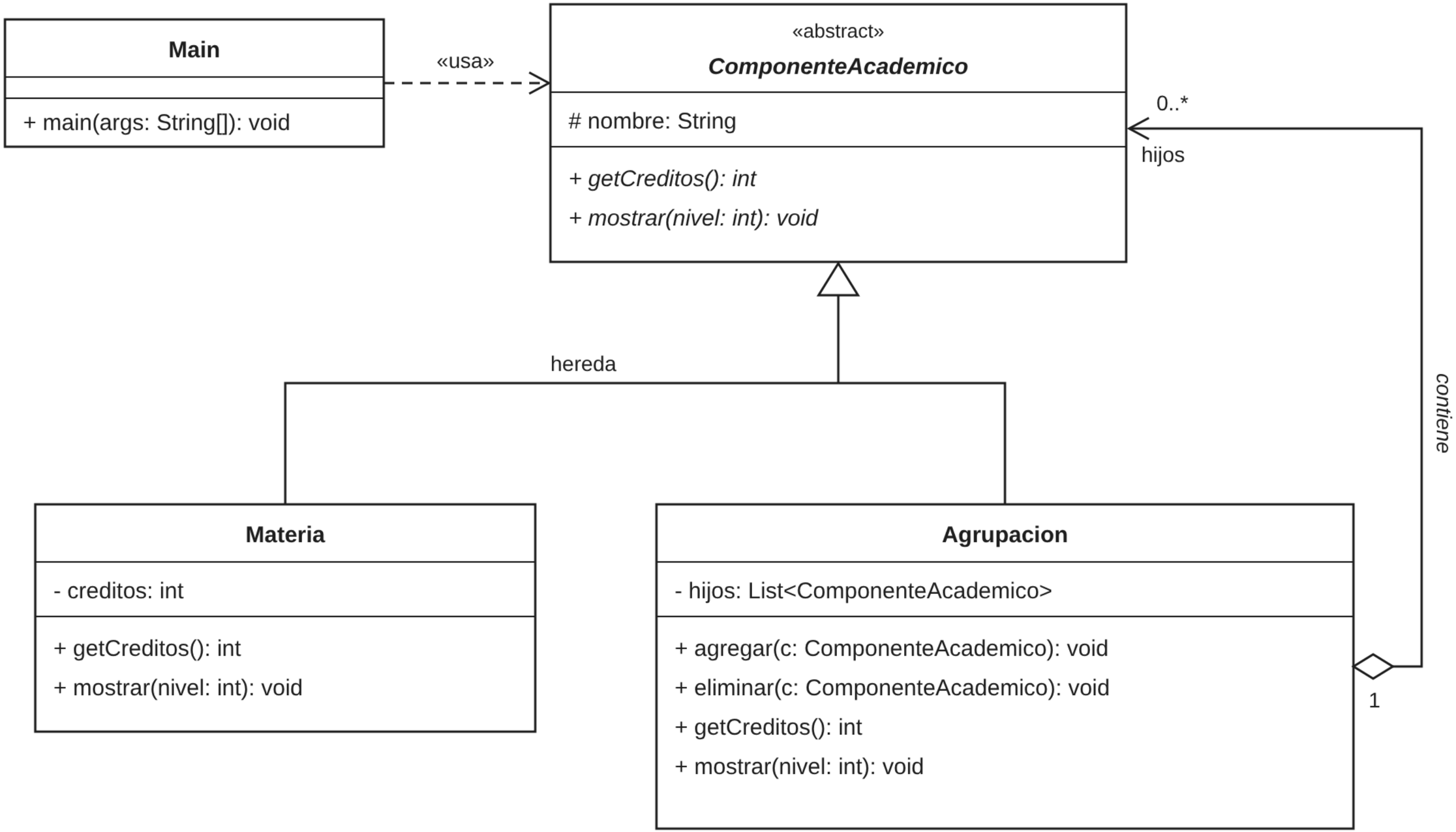
Un tipo nuevo obliga a modificar todo

SIN EL PATRÓN

```
class Materia { int creditos; }
class Agrupacion { List<Object> elementos; }

int calcularCreditos(Agrupacion a) {
    int total = 0;
    for (Object o : a.elementos) {
        if (o instanceof Materia m)
            total += m.creditos;
        else if (o instanceof Agrupacion sub)
            total += calcularCreditos(sub);
    }
    return total;
}
```

Diagrama UML



Con Composite

```
abstract class ComponenteAcademico {
    abstract int getCreditos();
    abstract void mostrar(int nivel);
}

class Materia extends ComponenteAcademico {
    private int creditos;
    int getCreditos() { return creditos; }
}

class Agrupacion extends ComponenteAcademico {
    private List<ComponenteAcademico> hijos;
    int getCreditos() {
        int total = 0;
        for (ComponenteAcademico c : hijos) {
            total += c.getCreditos();
        }
        return total;
    }
}
```

Es un componente

Materia y Agrupacion heredan de la misma clase: el cliente las trata igual.

Tiene componentes

La lista es del tipo común: guarda materias u otras agrupaciones.

Delega en sus hijos

Le pregunta a cada hijo y suma. Si el hijo es un grupo, hace lo mismo: recursión.

Antes vs. después

Antes (sin patrón)	Después (Composite)
List<Object>: cualquier cosa cabe	List<ComponenteAcademico>: solo el árbol
if / instanceof para saber el tipo	Polimorfismo: cada clase sabe responder
Un recorrido manual por operación	El grupo delega en sus hijos
Un tipo nuevo obliga a modificar código	Se agrega una clase sin tocar lo demás

El cliente pasa de recorrer el árbol a mano a una sola llamada: `plan.getCreditos()`

Solución

```
Agrupacion sem1 = new Agrupacion("Semestre 1");
sem1.agregar(new Materia("Cálculo Diferencial", 4));
sem1.agregar(new Materia("Programación I", 3));
sem1.agregar(new Materia("Introducción a la Ingeniería", 2));

Agrupacion electivas = new Agrupacion("Electivas");
electivas.agregar(new Materia("Ética", 2));
electivas.agregar(new Materia("Inglés I", 2));

Agrupacion sem2 = new Agrupacion("Semestre 2");
sem2.agregar(new Materia("Cálculo Integral", 4));
sem2.agregar(new Materia("Programación II", 3));
sem2.agregar(electivas);  // un grupo dentro de otro

Agrupacion plan = new Agrupacion("Plan Ingeniería de Sistemas");
plan.agregar(sem1);
plan.agregar(sem2);
plan.getCreditos()
```

Arma el árbol

Con agregar() mete materias o agrupaciones en cualquier grupo.

Anida sin límite

Electivas es un grupo dentro del Semestre 2.

Una sola llamada

Nunca pregunta el tipo: solo le pide al plan que se muestre.

EJEMPLO DEL REPOSITORIO · PLAN DE ESTUDIOS

Ventajas y desventajas en el ejemplo

✔ Ventajas

Una sola llamada

`plan.getCreditos()` suma todo el plan: **20 créditos**, sin recorrer a mano.

Grupos dentro de grupos

Electivas vive dentro de **Semestre 2** sin escribir código extra.

Tipo nuevo sin tocar nada

Un `Laboratorio` que herede de `ComponenteAcademico` funciona sin cambiar `Agrupacion` ni `Main`.

Lista segura

`List<ComponenteAcademico>`: ya no cabe cualquier `Object`.

✘ Desventajas

No es del todo uniforme

`agregar()` solo existe en `Agrupacion`: para armar el árbol hay que saber cuál es grupo.

Sin control de ciclos

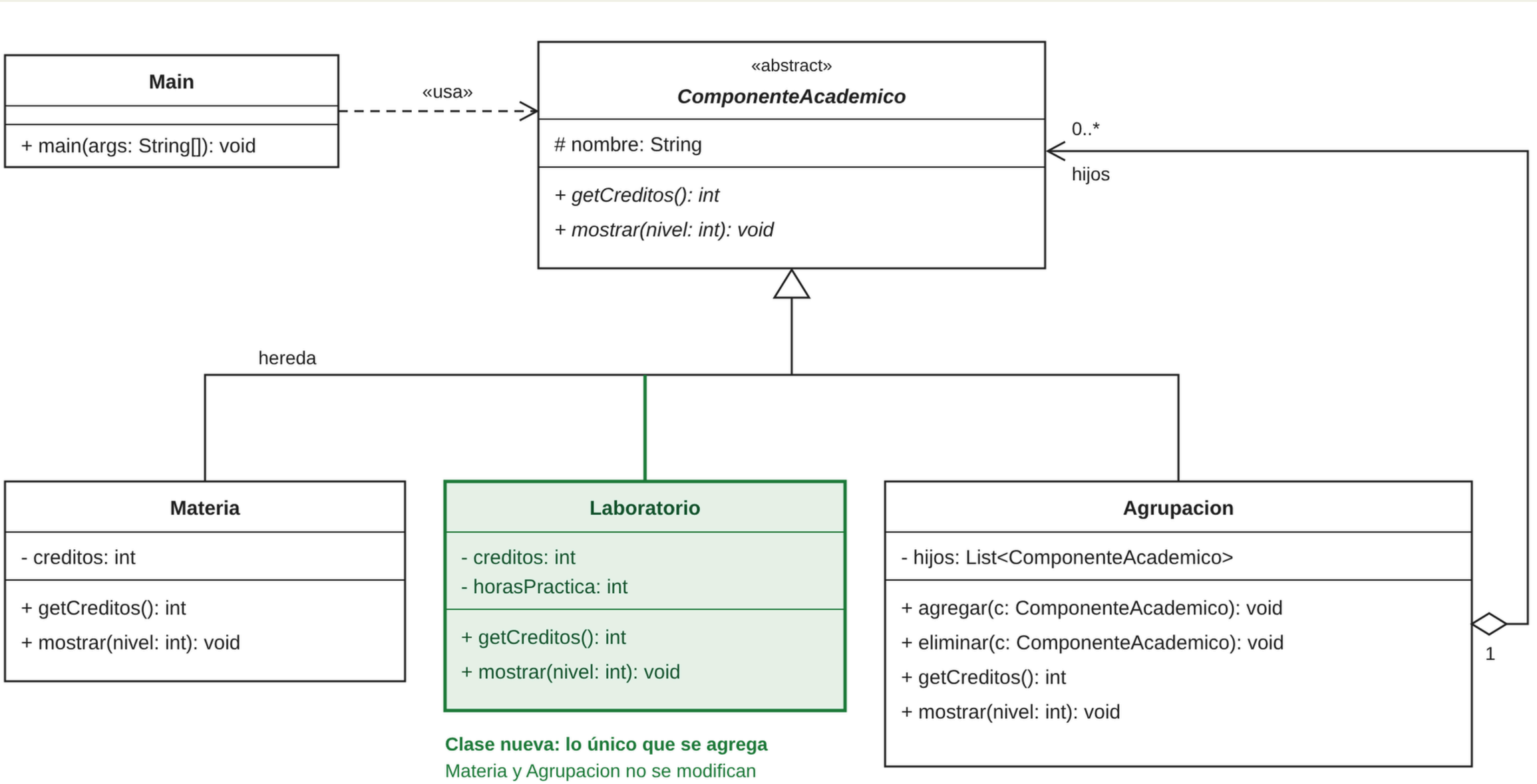
`semestre.agregar(semestre)` compila y al sumar da **StackOverflowError**.

Se recalcula todo

`mostrar()` llama `getCreditos()` en cada grupo: recorre el mismo subárbol varias veces.

Sin reglas del dominio

Se puede meter el Plan dentro de un Semestre o repetir una materia: nada lo valida.



Clase nueva: lo único que se agrega

Materia y Agrupacion no se modifican

EJEMPLO DEL REPOSITORIO

Stack y cómo ejecutar el repo

Lenguaje

Java 11 o superior (JDK) · POO: herencia, clase abstracta y polimorfismo

Sin librerías

Solo `java.util.List` y `ArrayList`

Editor

IntelliJ, NetBeans o VS Code con Extension Pack for Java

Repositorio

Git y GitHub · diagramas UML con Mermaid

TERMINAL · desde la raíz del repo

```
git clone https://github.com/Luis301311/patron-composite
cd patron-composite
javac -encoding UTF-8 -d out src/despues/*.java
java -cp out despues.Main
```

SALIDA

```
+ Plan Ingeniería de Sistemas (20 cr)
  + Semestre 1 (9 cr)
    - Cálculo Diferencial (4 cr) …
  + Semestre 2 (11 cr)
    - Cálculo Integral (4 cr) …
    + Electivas (4 cr)
      - Ética (2 cr)
      - Inglés I (2 cr)

Total de créditos: 20
```

APLICACIÓN EN EL PROYECTO

OpticaSync Pro: el menú lateral

Sistema de gestión de la óptica **JD Fashion**: Next.js + FastAPI + PostgreSQL.

El menú lateral es un **árbol**: secciones, opciones y submenús.

Cada rol ve un menú distinto: vendedor, admin, superadmin y biselador. Bisel y Laboratorio muestran **contadores** de pendientes.

Hay que tratar igual a una opción suelta y a un grupo de opciones.



OPTICASYNC PRO · SIN EL PATRÓN

Antes de aplicar Composite

components/sidebar.tsx · 563 líneas

```
{isBiselador ? (
  <> {/* otro menú completo para el biselador */} </>
) : (
  <ul>
    <li>
      <Link href="/" className={cn('flex items-center gap-3 …',
          pathname === '/' ? 'bg-sidebar-primary …'
                           : 'text-sidebar-foreground/70 …')}>
        <LayoutDashboard className="h-[18px] w-[18px]" />
        Dashboard
      </Link>
    </li>
    {hasAccess(userRole, 'admin') && (
      <li>
        <Link href="/proveedores" className={cn(/* igual */)}>
          <Users className="h-[18px] w-[18px]" />
          Proveedores
        </Link>
      </li>
    )}
    {/* … 16 bloques más, copiados */}
  </ul>
)}
```

Todo a mano

18 opciones copiadas; la clase de «activo» se repite **17 veces**.

Permisos regados

hasAccess() en 11 lugares del JSX.

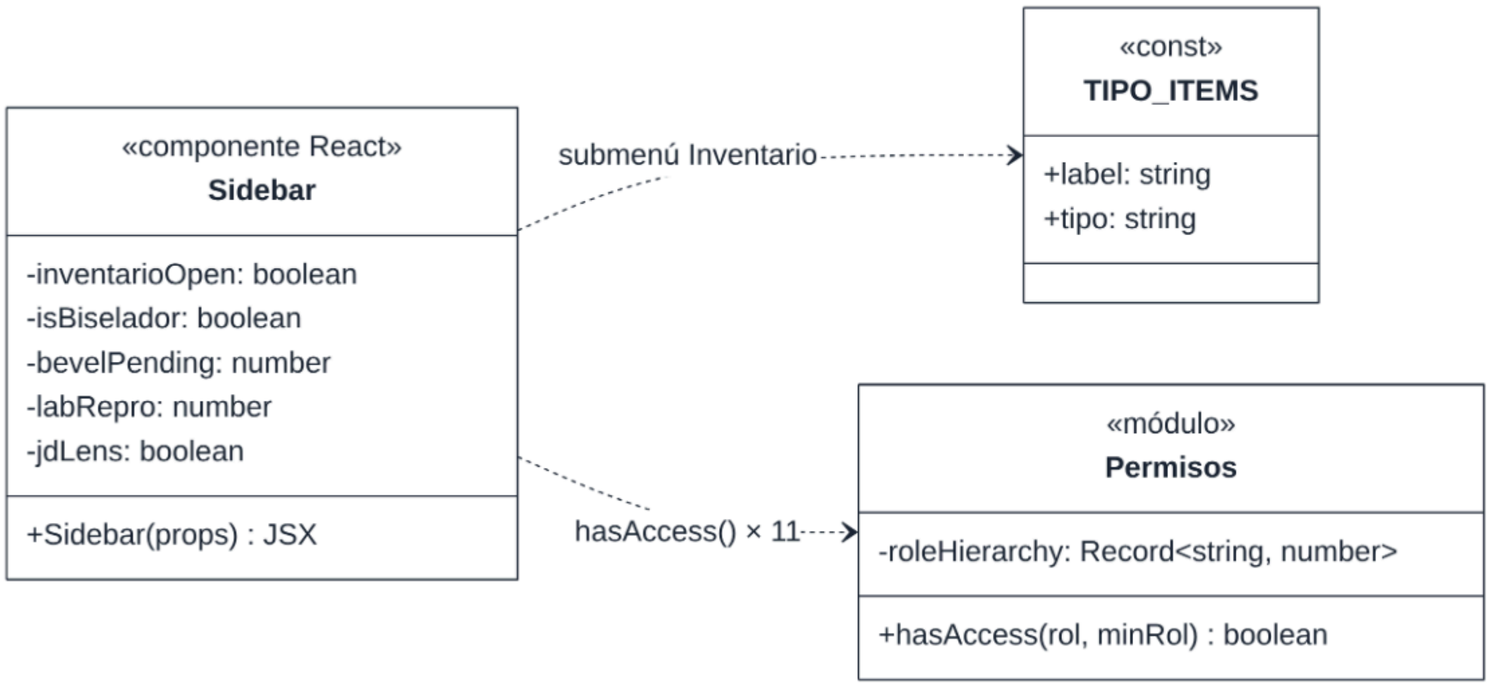
Casos especiales

El biselador tiene **otro menú aparte** y cada submenú su propio código.

Difícil de extender

Una opción nueva = copiar **~20 líneas** de JSX.

Diagrama de clases: antes



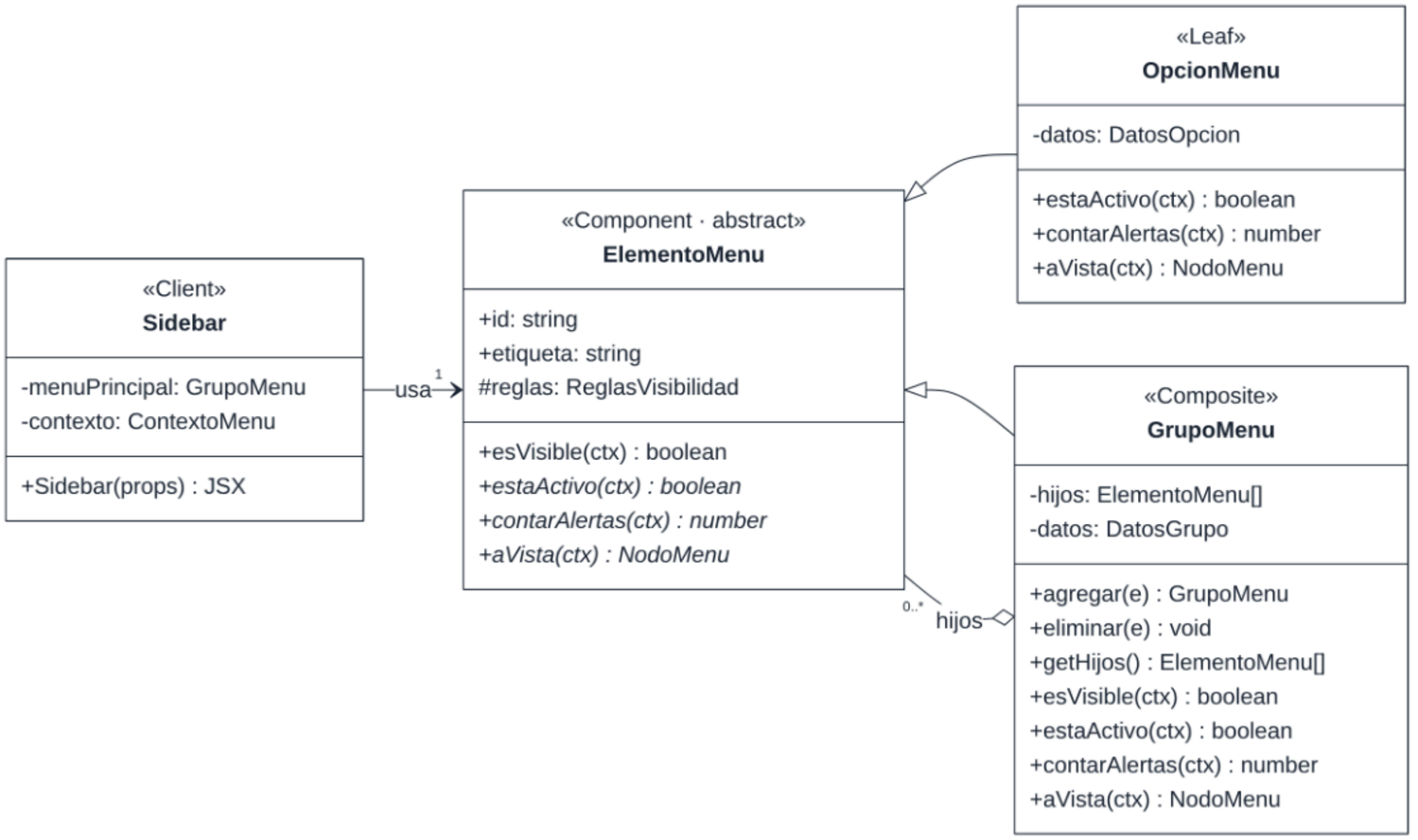
Una sola clase

Sidebar lo hace todo. No hay un tipo común para opción y submenú: el árbol solo existe en el JSX.

Notación UML

+ público
- privado · # protegido
cursiva = método abstracto
«…» = estereotipo / rol
⟶ asociación
⇢ dependencia
▷ herencia (generalización)
◇ agregación, con multiplicidad

Diagrama de clases: después



Tipo común + recursión

ElementoMenu es abstracta; **GrupoMenu** agrega 0..* ElementoMenu: grupos dentro de grupos.

Notación UML

+ público
- privado · # protegido
cursiva = método abstracto
«…» = estereotipo / rol
⟶ asociación
⇢ dependencia
▷ herencia (generalización)
◇ agregación, con multiplicidad

OPTICASYNC PRO · CON EL PATRÓN

Después de aplicar Composite

lib/menu

```
abstract class ElementoMenu {          // Component
  abstract estaActivo(ctx): boolean
  abstract contarAlertas(ctx): number
  abstract aVista(ctx): NodoMenu | null
}

class OpcionMenu extends ElementoMenu {  // Leaf
  contarAlertas(ctx) {
    return this.datos.alertas?.(ctx) ?? 0
  }
}

class GrupoMenu extends ElementoMenu {   // Composite
  private hijos: ElementoMenu[] = []

  agregar(...e: ElementoMenu[]) { this.hijos.push(...e); return this }

  contarAlertas(ctx) {
    return this.hijos
      .filter((h) => h.esVisible(ctx))
      .reduce((t, h) => t + h.contarAlertas(ctx), 0)
  }
}
```

Rol	Clase
Component	ElementoMenu
Leaf	OpcionMenu Dashboard, Clientes, Bisel…
Composite	GrupoMenu Administración, Inventario, JD Lens
Client	Sidebar

El grupo **suma** las alertas de sus hijos, se **resalta** si un hijo está activo y se **oculta** si no tiene hijos visibles.

OPTICASYNC PRO · RESULTADO

Mismo menú, mejor código

CLIENTE

```
// El árbol se declara una sola vez
new OpcionMenu('bisel', 'Bisel', {
  href: '/bisel', icono: Gem, rolMinimo: 'admin',
  alertas: (ctx) => ctx.alertas.bisel,
})

// sidebar.tsx (Client): sin if por rol ni por opción
const secciones = menuPrincipal.aVista(contexto)?.hijos
secciones.map((s) => <ElementoMenuVista nodo={s} />)
```

VERIFICACIÓN

```
HTML viejo vs. nuevo: idéntico en 24 casos
  (4 roles × 6 rutas)
Pruebas: 76/76 · TypeScript sin errores
```

	Antes	Después
sidebar.tsx	563 líneas	286 líneas
Opción nueva	Copiar ~20 líneas JSX	Un new OpcionMenu
Permisos	hasAccess × 11	rolMinimo por elemento
Biselador	Menú aparte	Una sección más
Submenús	Código propio c/u	GrupoMenu + vista recursiva
Pruebas	Ninguna	13 pruebas

Conclusiones

01

Sirve cuando el problema tiene forma de árbol parte-todo.

02

Su fuerza está en la interfaz común más la recursión.

03

El cliente queda simple y el sistema abierto a nuevos tipos.

04

Su costo: una interfaz muy general. No usarlo sin una jerarquía real.

Ejercicio Practico

DE QUÉ SE TRATA

Una biblioteca de música: una playlist puede contener canciones y otras playlists.

QUÉ SE PIDE

Calcular la duración total y contar las canciones de cualquier playlist, incluidas las sub-playlists.

RESTRICCIÓN

Prohibido usar instanceof o preguntar el tipo de un elemento.

